

COMSATS UNIVERSITY ISLAMABAD



DEPARTMENT OF COMPUTER SCIENCE

CSC336 - WEB TECHNOLOGIES

PropInsight: Web-Based Real Estate Analytics and Investment System Report

SUBMITTED TO:

Sir Aamir Shabbir Pare

SUBMITTED BY:

Khushnood Ahmad

SP24-BCS-051

Muhammad Umar

SP24-BCS-076

DATED:

5th May, 2026

1. Introduction

1.1 Overview

PropInsight is a web-based analytics platform designed to help buyers, investors, and analysts make data-driven real estate decisions. The system integrates property searching with interactive financial tools, such as ROI and mortgage calculators, and data visualizations for market trends.

1.2 Scope

The system provides:

- Property searching, filtering, and geographical mapping.
- Detailed property analytics and side-by-side comparison tools.
- Financial calculators for ROI and mortgage estimation.
- A responsive web interface built using HTML, CSS, JavaScript, and Laravel.

2. Overall Description

2.1 Product Perspective & Features

PropInsight functions as an independent analytical dashboard. Major features include:

- **Search & Listing:** Advanced filtering by price, location, and type.
- **Analytics:** Detail pages featuring image galleries and trend charts.
- **Comparison Engine:** Side-by-side tables for technical and financial metrics.
- **Map Explorer:** Interactive geographical property pins.

2.2 User Classes

1. **Guest User:** Can browse, search, and filter property listings.
2. **Registered User:** Access to comparison tools, saved favorites, and financial calculators.
3. **Administrator:** Manages property data, listings, and system maintenance.

2.3 Operating Environment

- **Platforms:** Web Browsers (Chrome, Firefox, Edge) on Windows, macOS, and Linux.
- **Tech Stack:** HTML/CSS/JS (Frontend), Laravel (Backend), MySQL (Database).

3. Functional Requirements

3.1 Landing & Listing Pages

- **Search Engine:** Users can search by location and view featured properties.
- **Filtering & Sorting:** Ability to refine results by price and specifications using a card-based UI.

3.2 Property Detail & Comparison

- **Analytics Dashboard:** Displays property specs alongside visual data charts.
- **Comparison Tool:** A side-by-side interface comparing prices, features, and projected ROI for multiple selections.

3.3 Financial Tools & Mapping

- **ROI/Mortgage Calculator:** Input fields for investment costs and interest rates with visual outcome graphs.
- **Map Explorer:** A map-based interface allowing users to explore properties geographically via interactive pins.

4. Non-Functional Requirements

- **Performance:** Pages must load within 3 seconds; support for concurrent users.
- **Security:** Secure authentication and data protection for registered accounts.
- **Usability:** Intuitive, responsive design optimized for desktop and mobile devices.
- **Reliability:** High system uptime with consistent data accuracy.

5. Future Enhancements

- AI-driven property recommendations based on user behavior.
- Integration with real-time real estate market APIs.
- Native mobile application (iOS/Android) version.

Part 2: Frontend Development & UI Design

2.1 Overview

The PropInsight frontend is built as a responsive web application designed for high-performance data visualization. It utilizes a modular component architecture to ensure a seamless experience across desktop and mobile devices, focusing on clarity for complex real estate metrics.

2.2 Technology Stack

- **Structure:** HTML5 (Semantic elements for SEO and accessibility).
- **Styling:** Bootstrap 5.3.2 & Custom CSS3 (Flexbox/Grid).
- **Interactivity:** Vanilla JavaScript (ES6+).
- **Icons:** FontAwesome for intuitive UI navigation.

2.3 User Interface Components

2.3.1 Navigation & Branding

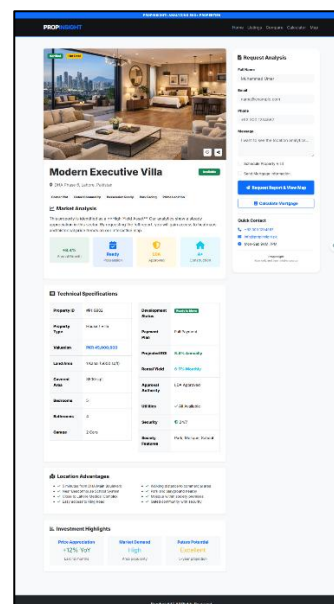
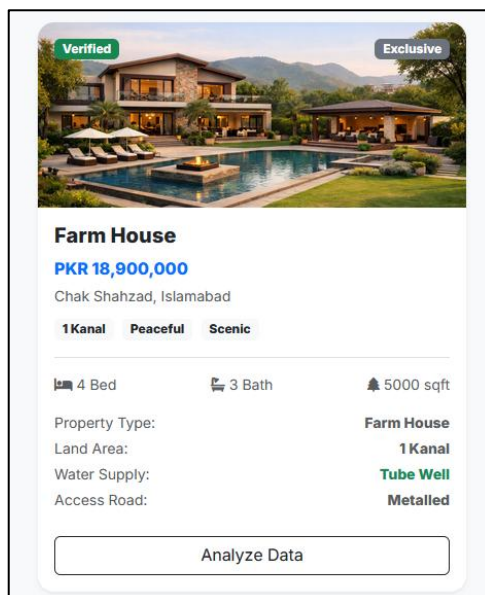
The system features a sticky Bootstrap navigation bar for consistent access to core tools.

- **Brand Identity:** "PROPINSIGHT" with a specific blue highlight on the suffix.
- **Navigation Links:** Home, Listings, Compare, Calculator, and Map.

2.3.2 Property Listing Architecture

Properties are displayed using a card-based layout. Each card contains:

- **Badges:** Real-time status indicators (e.g., "Verified," "Hot Deal").
- **Metric Grid:** Quick-view icons for bedrooms, bathrooms, and square footage.
- **Actionable Data:** A dedicated "Analyze Data" button linking to deep-dive analytics.



2.3.3 Analysis & Reporting UI

The interface includes specialized components for professional users, such as:

- **Feature Cards:** Icons and summaries for generating PDF reports.
- **Comparison Matrix:** Side-by-side tables with responsive wrappers.

2.4 Design & Styling (CSS)

The visual language is defined by a clean, professional aesthetic using the following principles:

- **Color Palette:** A dark branding base (#121416) contrasted with a primary "Action Blue" (#0d6efd).
- **Box Model:** Property cards utilize a 16px border-radius and soft shadows.
- **Interactivity:** Hover effects include a subtle lift (`translateY`) to improve user engagement.
- **Typography:** Implementation of the "Inter" sans-serif font for maximum readability of financial figures.

2.5 Interactive Logic (JavaScript)

2.5.1 Real-Time Market Ticker

A dynamic header ticker provides live updates on market rates (e.g., Gold rates, Interest rates) using a `setInterval` timer.

- **Interval:** Updates every 4 seconds.
- **Implementation:** DOM manipulation targeting the `#market-ticker` element.



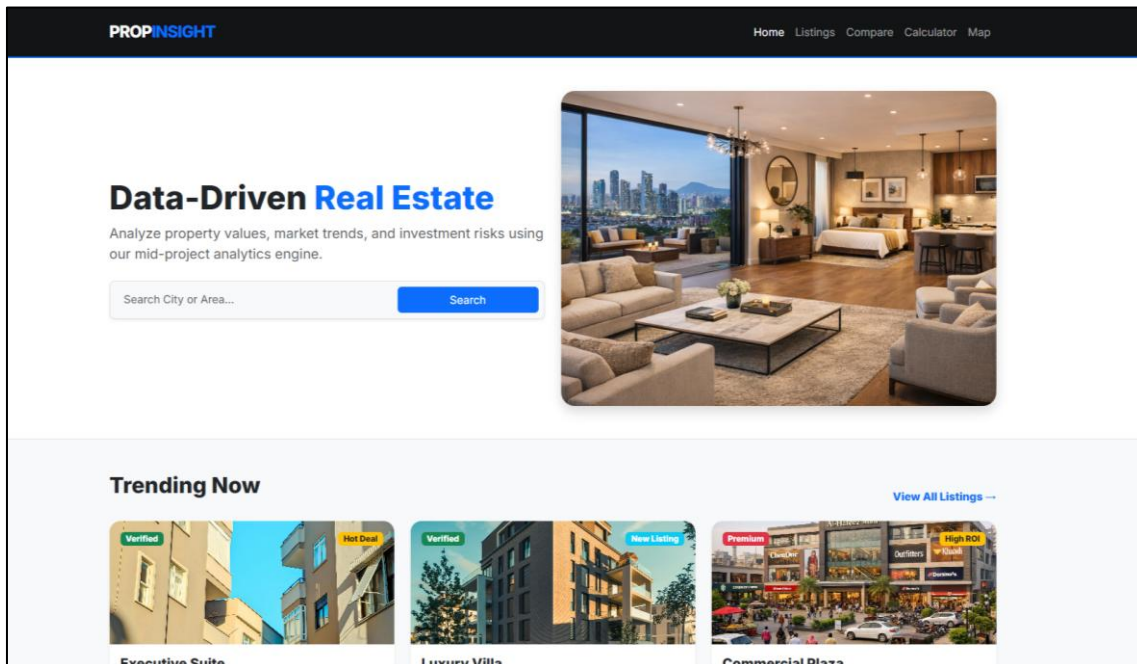
2.5.2 Mortgage Calculation Engine

The mortgage tool uses custom logic to process user inputs and provide immediate visual feedback.

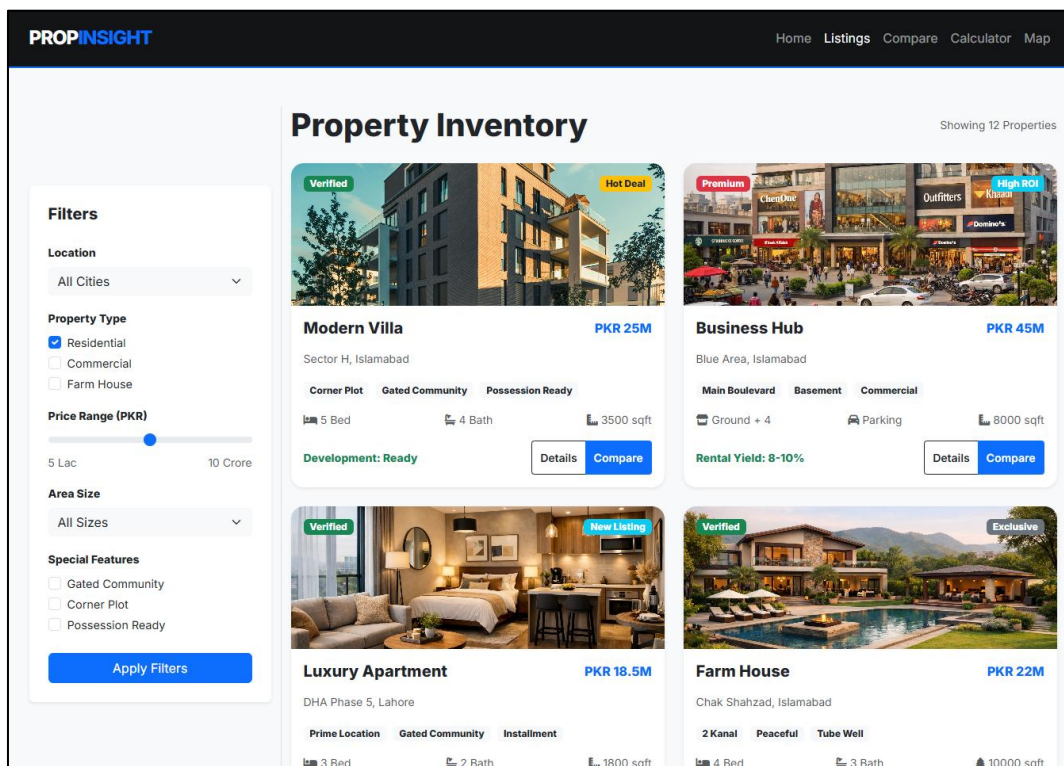
- **Formula:** $(\text{Property Price} - \text{Down Payment}) / \text{Loan Tenure in Months}$.
- **Features:** Automated PKR currency formatting and a "Fade-In" animation for result display.

2.6 Visual Documentation (Screenshots)

Hero Section:



Listings Page:



Comparison Page:

NEW LISTINGS IN GULBERG: +12 TODAY			
PROPINSIGHT			
Home Listings Compare Calculator Map			
Property Comparison Matrix			
Side-by-side analysis of your selected investments with Pakistan-specific insights.			
FEATURES	LUXURY VILLA	MODERN APARTMENT	CORPORATE PLAZA
Price (PKR)	45,000,000	18,500,000	95,000,000
Location	DHA Phase 6, Lahore	Gulberg III, Lahore	Blue Area, Islamabad
Area Size	1 Kanal (5000 sqft)	1250 sqft	4500 sqft
Property Type	House	Apartment	Commercial
Pakistan Features	Corner Plot Gated Possession Ready	Prime Location Installment Under Construction	Main Boulevard Basement High ROI
Development Status	Ready to Move	Under Construction	Booked
Payment Plan	Full Payment	Installment Available	Full Payment
Rental Yield	High (8-9%)	Moderate (4-5%)	Very High (8-10%)
Approval Authority	LDA Approved	LDA Approved	CDA Approved
Utilities	✓ All Available	✓ All Available	✓ All Available
Security	24/7	24/7	24/7
Investment Score	85%	70%	95%
Action	View Data	View Data	View Data

Mortgage Calculator:

PROPINSIGHT

Home Listings Compare Calculator Map

Pakistan Real Estate Investment Calculator

Calculate mortgage payments, ROI, and investment returns for Pakistani properties

Mortgage & Investment Estimator

Calculate your monthly installments and investment returns instantly.

Property Price (PKR)

PKR
e.g. 15000000

Enter total property price in Pakistani Rupees

Loan Tenure (Years)

15 Years

Maximum 25 years in Pakistan

Property Type

Residential

Affects loan terms and rates

☐ Calculate ROI and Investment Returns

Down Payment (%)

e.g. 20

Typically 20-30% for Pakistani banks

Interest Rate (%)

11.5

Current KIBOR + spread (approx 11-13%)

City

Karachi

Location affects property values

Calculate Investment

Part 3: Backend Architecture & Implementation

3.1 System Architecture

PropInsight is engineered using the **Laravel 11** framework, adhering to the **Model-View-Controller (MVC)** design pattern. This architecture ensures a clean separation of concerns, where business logic is isolated from the data layer and the user interface.

3.1.1 Core Backend Stack

- **Language:** PHP 8.2+
- **Framework:** Laravel 11 (utilizing Eloquent ORM and Blade Templating)
- **Database:** MySQL 8.0/MariaDB (Relational data mapping)
- **Web Server:** Nginx/Apache with PHP-FPM
- **Session Management:** Laravel Session Driver for cross-page persistence (Comparison Tool)

3.1.2 Request Lifecycle

The application follows a standard Laravel request lifecycle:

1. **Entry:** All requests enter through `public/index.php`.
2. **Routing:** The `routes/web.php` file directs requests to specific Controllers.
3. **Middleware:** Requests pass through the `VerifyCsrfToken` and `StartSession` middleware groups.
4. **Controller Execution:** The business logic is processed, interacting with Models.
5. **Response:** A Blade view is rendered and returned to the client.

3.2 Database Schema & Data Modeling

The system relies on a normalized relational database. Data integrity is maintained through foreign key constraints, indices for search optimization, and Laravel's Eloquent relationships.

3.2.1 Entity Relationship Summary

- **City Model:** Stores geographical locations.
 - `public function properties() { return $this->hasMany(Property::class); }`
- **PropertyType Model:** Categorizes listings (Residential, Commercial, etc.).
 - `public function properties() { return $this->hasMany(Property::class); }`
- **Property Model:** The central entity.
 - `public function city() { return $this->belongsTo(City::class); }`
 - `public function type() { return $this->belongsTo(PropertyType::class); }`

3.2.2 Database Migrations

The schema is version-controlled using Laravel Migrations. The `properties` table includes specialized columns for:

- `decimal('price', 15, 2)` to handle large currency values accurately.
- `string('slug')->unique()` for SEO-friendly URLs.
- `spatialIndex` (optional) for map-based coordinate lookups.

3.3 Core Logic & Controllers

3.3.1 Financial Calculation Engine

The `CalculatorController` handles complex financial projections. It implements server-side validation to prevent division-by-zero errors and ensures all inputs fall within realistic real-estate parameters.

Detailed Implementation:

```
public function calculate(Request $request) {
    // Phase 1: Input Validation
    $request->validate([
        'price' => 'required|numeric|min:100000',
        'down_payment' => 'required|numeric|min:0|max:100',
        'tenure' => 'required|integer|min:1|max:25',
        'interest_rate' => 'required|numeric|min:0|max:30',
    ]);

    // Phase 2: Processing Logic
    $price = $request->input('price');
    $downPaymentAmount = $price * ($request->input('down_payment') /
100);
    $loanAmount = $price - $downPaymentAmount;

    $totalMonths = $request->input('tenure') * 12;
    $monthlyRate = $request->input('interest_rate') / 100 / 12;

    if ($monthlyRate == 0) {
        $monthlyInstallment = $loanAmount / $totalMonths;
    } else {
        // Standard Mortgage Amortization Formula: M = P [ i(1 + i)^n ] /
[ (1 + i)^n - 1]
        $monthlyInstallment = ($loanAmount * $monthlyRate) / (1 - pow(1 +
$monthlyRate, -$totalMonths));
    }

    // Phase 3: Investment Analytics (ROI)
    // Projects 5% annual appreciation and rental yield
    $annualIncome = $price * 0.05;
    $roi = ($price > 0) ? ($annualIncome / $price) * 100 : 0;
}
```

3.3.2 Dynamic Search & Filtering Engine

The `PropertyController` utilizes a "Fluent" query builder pattern. This approach allows for highly flexible search queries where parameters are only added if the user provides them.

Search Implementation:

```
public function index(Request $request) {
    // Validation for Price Logic
    $request->validate([
        'min_price' => 'nullable|numeric|min:0',
        'max_price' => 'nullable|numeric|gt:min_price',
    ]);
    // Initialize Query with Eager Loading (Optimization)
    $query = Property::with(['city', 'type']);
    // Dynamic Filter Binding: Add constraints only if present in request
    if ($request->filled('city')) {
        $query->whereHas('city', fn($q) => $q->where('slug', $request->city));
    }
    if ($request->filled('type')) {
        $query->whereHas('type', fn($q) => $q->where('slug', $request->type));
    }
    if ($request->filled('search')) {
        $query->where(function($q) use ($request) {
            $q->where('title', 'like', '%' . $request->search . '%')
                ->orWhere('description', 'like', '%' . $request->search . '%');
        });
    }
    // Sorting and Pagination
    $properties = $query->latest()->paginate(9)->withQueryString();
    return view('pages.listings', compact('properties'));
}
```

3.3.3 Session-Based Comparison Tool

The `CompareController` utilizes Laravel's `session()` helper to store up to 3 property IDs. This "Queue" logic ensures that when a 4th property is added, the oldest one is removed.

Comparison Management Logic:

```
public function add(Request $request) {
    $propertyId = $request->input('property_id');
    $compareIds = session()->get('compare_properties', []);

    if (!in_array($propertyId, $compareIds)) {
        // Maintain a strict 3-slot limit
        if (count($compareIds) >= 3) {
            array_shift($compareIds); // Remove oldest entry
        }
        $compareIds[] = $propertyId;
        session()->put('compare_properties', $compareIds);
    }

    return redirect()->route('compare.index');
}
```

3.4 Templating & View Logic

3.4.1 Deterministic Asset Handling

To maintain UI consistency, PropInsight uses a "Deterministic Fallback" system. If a property does not have an uploaded image, the system calculates a fallback image based on the loop index, ensuring variety in the gallery.

```
@php
    // Collection of verified high-res placeholders
    $placeholders = ['p1.jpg', 'p2.jpg', 'penthouse.png', 'plaza.png',
'studio.png'];

    // Deterministically pick image based on property iteration
    $imagePath = 'images/' . $placeholders[$loop->index %
count($placeholders)];

    // Check for user-uploaded featured image
    if ($property->featured_image && file_exists(public_path('images/' .
$property->featured_image))) {
        $imagePath = 'images/' . $property->featured_image;
    }
@endphp
title }}" />
```

3.5 Security & Optimization

3.5.1 Middleware & Security

- **CSRF Protection:** Every state-changing request (POST, PUT, DELETE) is validated via the `@csrf` directive to prevent cross-site request forgery.
- **SQL Injection Prevention:** The use of Eloquent ORM ensures that all database queries are prepared statements, effectively neutralizing SQLi attacks.
- **XSS Protection:** Blade's `{{ $var }}` syntax automatically escapes output. For raw HTML (e.g., descriptions), strict sanitization is applied.

3.5.2 Performance Tuning

- **N+1 Query Prevention:** The system uses Eager Loading via the `with()` method to fetch cities and types in a single query rather than one query per property card.
- **Database Indexing:** The `slug` and `price` columns are indexed to ensure sub-millisecond search results even as the database grows.
- **Query Pagination:** Results are limited to 9 per page to reduce DOM weight and server memory usage.